

Lab 3: Component Modelling & Architectural Pattern Selection

Database Query Performance Profiler

Field	Details
Student Name	Srijan Akshit
SRN	PES1UG24AM284
Domain	Developer Tools & IT Operations
Assigned Stakeholders	Developer Tools & IT Operations, Database Administrator, Backend Lead
Lab	Lab 3 — Component Modelling & Architectural Pattern Selection

1. Scenario Understanding & Requirement Analysis

1.1 Problem Statement

The Database Query Performance Profiler is a developer and IT-operations tool designed to capture database query execution information, measure query performance, analyze slow or expensive queries, store historical performance metrics, and present useful results to technical stakeholders. The system helps developers, database administrators, and backend leads identify query bottlenecks while keeping profiling overhead low.

1.2 Key Functional Requirements

- Capture database query execution information and performance metrics.
- Measure query execution time and related performance indicators.
- Analyze collected queries to identify slow, expensive, or frequently executed queries.
- Store query-performance metrics and historical profiling information.
- Provide a backend API for retrieving profiler results.
- Display performance results and trends through a monitoring dashboard.

1.3 Key Non-Functional Requirements & Challenges

- Performance: Profiling should introduce minimal overhead to the monitored database/application.
- Scalability: The profiler should handle a high volume of query metrics and traffic spikes.
- Availability: A failure in one profiling capability should not unnecessarily stop the entire monitoring workflow.
- Security: Query text, database metadata, credentials, and stored performance information must be protected.
- Maintainability: Individual profiling capabilities should be independently modified, deployed, and scaled.

2. Architectural Style Analysis

The three architectural styles specified in the lab handout were evaluated for the Database Query Performance Profiler.

Style	Pros for this system	Cons for this system
Layered Architecture	Clear separation of presentation, business logic, and data; simple to understand and implement.	Different profiler workloads remain coupled; a spike in query collection or analysis can require scaling unrelated modules.
Microservices Architecture	Query collection, analysis, storage, API, and dashboard can scale and fail independently; supports service-level security boundaries and independent deployment.	Higher operational complexity and inter-service communication overhead; requires careful service availability and data-consistency design.
Client-Server Architecture	Centralized control and straightforward deployment; easy to manage for a small user base.	Central server can become a performance bottleneck and single point of failure; high-volume profiling workloads are harder to scale independently.

3. Architecture Selection

Selected Architecture: Microservices Architecture

“We chose the Microservices Architecture for the Database Query Performance Profiler.” The profiler consists of distinct capabilities for query collection, performance analysis, data storage, API access, and visualization. These capabilities have different workload characteristics and operational needs. Microservices allow each capability to be built, deployed, secured, and scaled independently, making the architecture suitable for a performance-monitoring system.

4. Component Identification

Five core components were identified, together with the external Database System being monitored.

#	Component	Responsibility
1	Query Collector	Captures query execution information and forwards query-performance metrics for analysis.
2	Query Analyzer	Analyzes query metrics and identifies slow, expensive, or frequently executed queries and performance trends.
3	Performance Data Store	Stores raw/processed query-performance metrics and historical profiling data.
4	Profiler API	Provides controlled API access to profiler results for dashboards and developer/operations tools.
5	Monitoring Dashboard	Presents query-performance metrics, slow-query findings, and historical trends to technical stakeholders.

5. UML Component Diagram

The following diagram shows the five core profiler components, the external Database System, the three assigned stakeholders, and labelled interfaces using UML-style provided and required interface notation. The connections represent the main flow of query-performance information through the profiler.

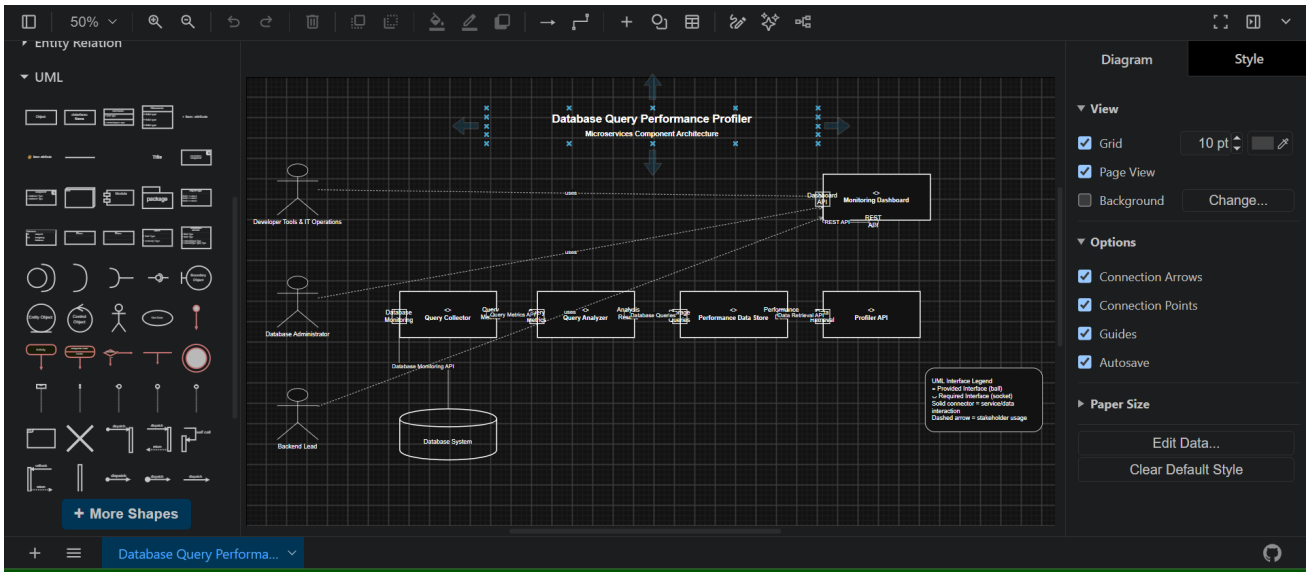


Fig. 1 — Database Query Performance Profiler component architecture

5.1 Interface Summary

Interface	Provider → Requirer	Purpose
IDatabaseMonitoring	Database System → Query Collector	Supplies query execution information and performance metrics to the collector.
IQueryMetrics	Query Collector → Query Analyzer	Transfers captured query metrics for performance analysis.
ISorePerformance	Query Analyzer → Performance Data Store	Stores analyzed query-performance results and historical metrics.
IDataRetrieval	Performance Data Store → Profiler API	Provides stored profiling results for controlled API retrieval.
IDashboardAPI	Profiler API → Monitoring Dashboard	Delivers profiler results and performance trends to the dashboard.

5.2 Requirement Verification

- At least 5 components: satisfied with five core components.
- At least 4 interfaces: satisfied with five labelled interfaces.
- Provided/required interfaces: represented using ball/socket-style notation in the diagram.
- Data flow and interactions: shown from Database System through collection, analysis, storage, API, and dashboard.
- Stakeholders: Developer Tools & IT Operations, Database Administrator, and Backend Lead are shown as dashboard users.

6. Written Justification

“We chose the Microservices Architecture for the Database Query Performance Profiler.”

Architectural Choice: The system is organized as independently deployable services for query collection, analysis, performance-data storage, API access, and dashboard visualization. Separating these responsibilities makes the profiler easier to scale and maintain.

Two Scenario-Related Reasons:

- Independent scaling for uneven workloads: Query collection may continuously receive a large volume of metrics, while analysis and dashboard traffic can vary independently. Microservices allow high-load services such as the Query Collector and Query Analyzer to be scaled without scaling the entire profiler.
- Fault isolation for monitoring: If the dashboard or API temporarily fails, query collection and analysis can continue operating. Separating the capabilities reduces the chance that one component failure stops the complete monitoring pipeline.

Security Advantage: Microservices create clear service boundaries around sensitive database information. Access to query data can be restricted to authorized services and API paths, while the dashboard receives only the information required by the user's role. Authentication, authorization, and secure communication can therefore be applied at service boundaries.

Performance Benefit: Independent scaling allows high-load components to receive additional resources without scaling low-load components unnecessarily. The Query Collector and Query Analyzer can be horizontally scaled when query-monitoring volume increases, helping maintain low processing latency and reducing infrastructure waste.

Conclusion: Microservices is the most suitable choice among the three evaluated styles because the profiler combines continuous collection, computational analysis, historical storage, API access, and visualization with different workload characteristics. The selected architecture supports scalability, fault isolation, performance, security, and maintainability.

7. Final Deliverable Verification

- Student credentials correctly recorded as Srijan Akshit, SRN PES1UG24AM284.
- Topic correctly recorded as Database Query Performance Profiler.
- Microservices Architecture selected and justified.
- Five core UML components and five interfaces included.
- Security and performance implications documented.
- Component diagram included in the report.